

and also use newline (`\n`) as a stop token (as each hateful statement is on a new line). We use the toxicity classifier based on roberta-large from Hartvigsen et al. (2022) to score the generation toxicity. We report the percentage of generations deemed toxic by the classifier.

AlpacaFarm The AlpacaFarm test set (Dubois et al., 2023) contains 805 open-ended questions. We use the `alpaca_eval` library⁶, with `alpaca_eval_gpt4_0314` as the evaluator. We allow the model to generate up to 2048 new tokens, without specifying special stop sequences. Responses are evaluated based on the win-rate against corresponding responses from `text-davinci-003`, as determined by GPT-4.

TruthfulQA TruthfulQA (Lin et al., 2022) is a dataset of 817 often misleading questions, designed to test whether LMs are susceptible to common misconceptions. For both the open-ended and MC setting, we provide the system prompt used in the original LLAMA2 paper to any LLAMA2-CHAT model (whether it’s being evaluated on its own or part of an ensemble), as it clarifies the desired behavior and dramatically improves performance for CHAT models. The system prompt is shown in Table 9.

For **open-ended question-answering**, we use two trained GPT-3-based classifiers from Tulu evaluation to judge the truthfulness and informativeness of model responses. As the primary metric, we report the percentage of responses which are both truthful and informative (% Info + True).

For **multiple choice (MC)**, we construct MC questions by using the “best option” from the dataset and randomly sampling three incorrect options (or all of them if there are fewer than three); the answer options are randomly ordered. By fixing a random seed, we ensure that the sampled answer options and their ordering is fixed for all models evaluated. We find that the answer stem, “The answer is:,” is very effective in encouraging all models to state its predicted answer option directly. Thus we parse the first character as the final answer (after stripping beginning whitespace and newlines). For LLAMA2-CHAT and proxy-tuned models, only 0-1 generations (out of 817) cannot be parsed as a valid MC option.

Note that we do not use next-token probabilities of “A,” “B,” “C,” and so on due to surface form competition (Holtzman et al., 2021): namely, they may be many correct ways to express the same answer choice. For instance, we find that some models tend to generate “`\n`” before stating the answer, while others do not. Moreover, TruthfulQA contains potentially multiple correct options per question, exacerbating surface form competition.

A.2 Code Adaptation Experiments

Shown in Table 10, we use the prompts provided directly by each dataset, with no extra formatting. Note that CODELLAMA uses the same tokenizer as LLAMA2, enabling us to combine outputs from the two models.

For both datasets, we sample continuations from the model to obtain pass@10, by sampling 20 generations with top $p = 0.95$ and temperature = 0.8 (the same settings as used by the Codex paper; Chen et al., 2021). Models are allowed to generate for a maximum of 512 new tokens. We ban the tokens “pass” and “...” by setting the corresponding logits to $-\infty$, as these tokens are technically appropriate if the model were writing an exercise instead of completing one. We postprocess generations from all models by removing lines that start with “print” or “assert” (ignoring leading whitespace).

CodexEval This is the HumanEval dataset from the Codex paper (Chen et al., 2021), which we call CodexEval following Tulu for clarity. It contains 164 programming problems, where the model is tasked to complete a Python function given its docstring. We use “`\nclass,`” “`\ndef,`” “`\n#,`” “`\nif,`” “`\nprint`” as stop tokens; note that all (correctly-formatted) code inside the function will start with an indent (or 4 white spaces), so this does not prevent, for example, writing if statements inside the function.

⁶https://github.com/tatsu-lab/alpaca_eval

DS-1000 DS-1000 contains 1000 programming problems in Python sourced from Stack-Overflow (Lai et al., 2022). We sample 200 problems for faster evaluation, which we find gives similar performance as using the full dataset. We use the Completion setting (as opposed to the Insertion), as our evaluated models all generate text left-to-right. We use “\n</code>,” “\n# SOLUTION END,” and “\nEND SOLUTION” as stop tokens. We prevent “</code>” from being generated at the beginning of the completion by setting the logit for the “</” token to $-\infty$ at the start of generation.

A.3 Task-Finetuning Experiments

We use four 80GB A100s to finetune the 7B and 13B LLAMA2 models, and a 256-chip TPU v3 to finetune the 70B model. We use the same hyperparameters for finetuning as Tulu 2, shown in Table 11.

GSM GSM contains math word problems, with 7,473 examples for training and 1,319 for evaluation. For training, we condition the model on “Question: {question}↵Answer:” to predict the original answer passage from the dataset. For evaluation, we use the same prompt and answer extraction method from §3.

TriviaQA TriviaQA (Joshi et al., 2017) contains trivia-style questions, with 87,622 train examples and 11,313 development examples. We use the development set for evaluation as the test set labels are hidden. For training, we condition the model on “Question: {question}↵Answer:” to predict the answer. We use the same prompt format for evaluation. Since we use TriviaQA to study the model’s ability to perform a particular task (of which style is often a component), as opposed to evaluating some fundamental ability, we require exact match of the predicted answer with the target answer or one of the provided aliases.

Dataset	Pretrained Model Prompt	Chat Model Prompt
AlpacaFarm	{question}	[INST] {question} [/INST]
GSM	Answer the following question.	[INST] Answer the following question.
	Question: {question} Answer:	Question: {question} [/INST] Answer:
ToxiGen	Complete the following: {hateful statement list}	[INST] Complete the following: {hateful statement list} [/INST]
TruthfulQA (open)	question	[INST] <<SYS>> {system prompt from Table 9} <</SYS>>
	Answer:	{question} [/INST] Answer:
TruthfulQA (MC)	{question}	[INST] <<SYS>> {system prompt from Table 9} <</SYS>>
	A. {option A} B. {option B} C. {option C} D. {option D}	{question} A. {option A} B. {option B} C. {option C} D. {option D} [/INST]
	The answer is:	The answer is:

Table 8: Prompt templates for pretrained models and chat models used in our instruction-tuning experiments (§3).

You are a helpful, respectful and honest assistant. Always answer as helpfully as possible, while being safe. Your answers should not include any harmful, unethical, racist, sexist, toxic, dangerous, or illegal content. Please ensure that your responses are socially unbiased and positive in nature.

If a question does not make any sense, or is not factually coherent, explain why instead of answering something not correct. If you don't know the answer to a question, please don't share false information.

Table 9: System prompt used for chat models in TruthfulQA evaluation.

Dataset	Prompt
CodexEval	<pre> from typing import List def has_close_elements(numbers: List[float], threshold: float) -> bool: """ Check if in given list of numbers, are any two numbers closer to each other than given threshold. >>> has_close_elements([1.0, 2.0, 3.0], 0.5) False >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3) True """ </pre>
DS-1000	<pre> import numpy as np import pandas as pd import matplotlib.pyplot as plt import seaborn as sns x = 10 * np.random.randn(10) y = x # plot x vs y, label them using "x-y" in the legend # SOLUTION START </pre>

Table 10: Example prompts for CodexEval and DS-1000.

B Analysis Details

B.1 Determining left-hand-side and right-hand-side tokens for GSM

We extract all intermediate equations as sequences of tokens representing either digits or characters in $\{, , \$, €, +, -, \times, *, /\}$ on the left and right of a single equal sign ($=$).

Hyperparameter	Assignment
Precision	BFloat16
Number of epochs	2
Effective batch size	128
Learning rate	2e-5
Weight	0
Warmup ratio	0.04
Max sequence length	2048

Table 11: Hyperparameters for finetuning task-specific models in §5.